Data link control

Maria Kihl

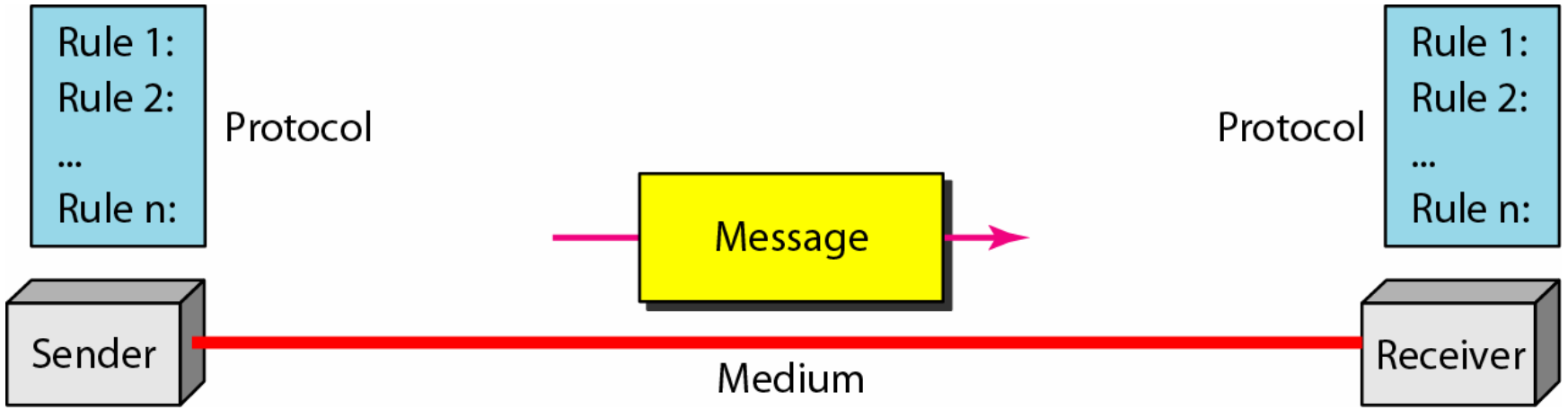
LUND
UNIVERSITY

Book chapters

Forouzan: 1.4, 10.1, 10.3-5, 11.1-5

Kihl: 4.1-4.3

Data communication

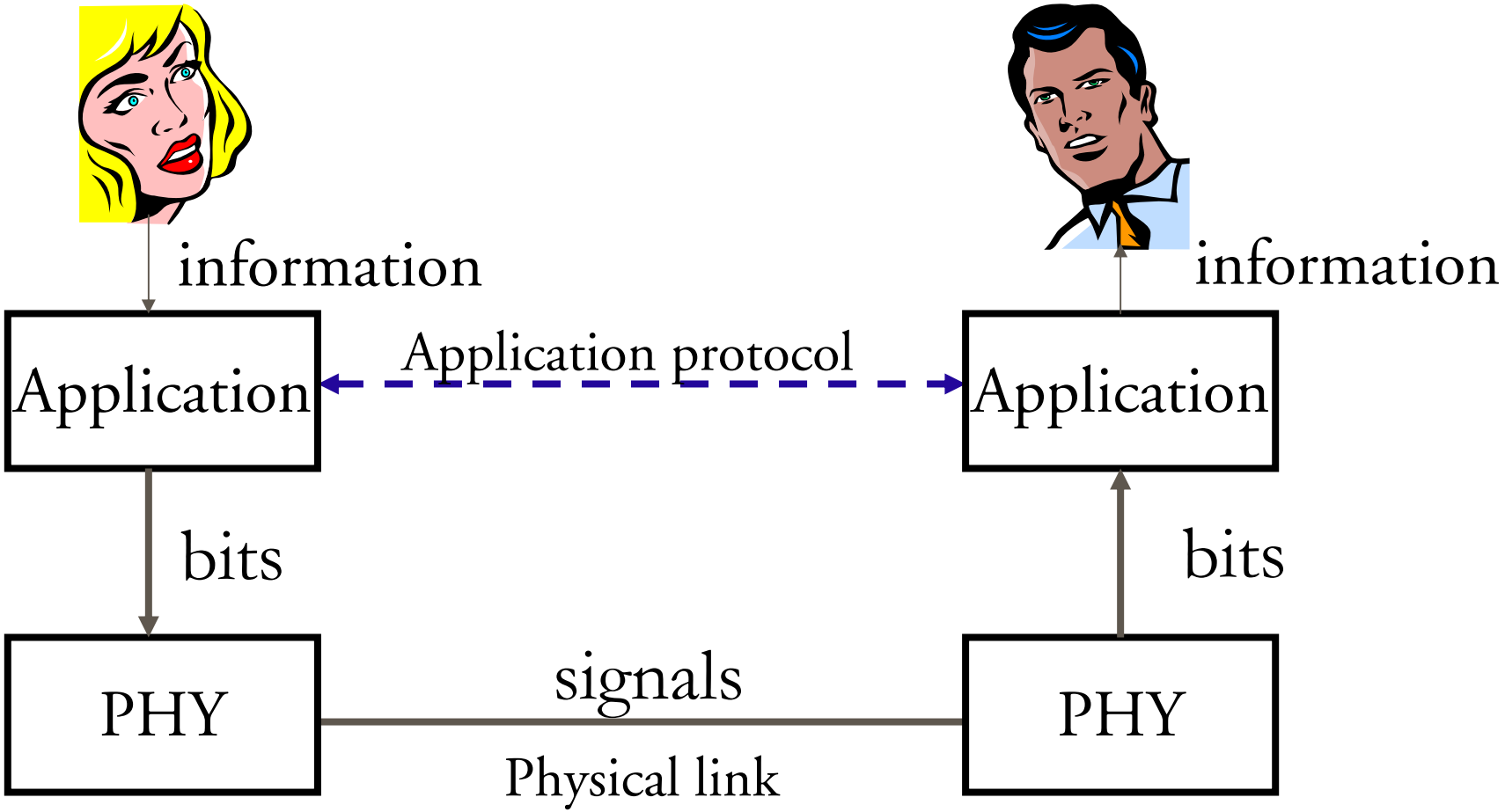


Data packets

Data is divided into packets before transmission.

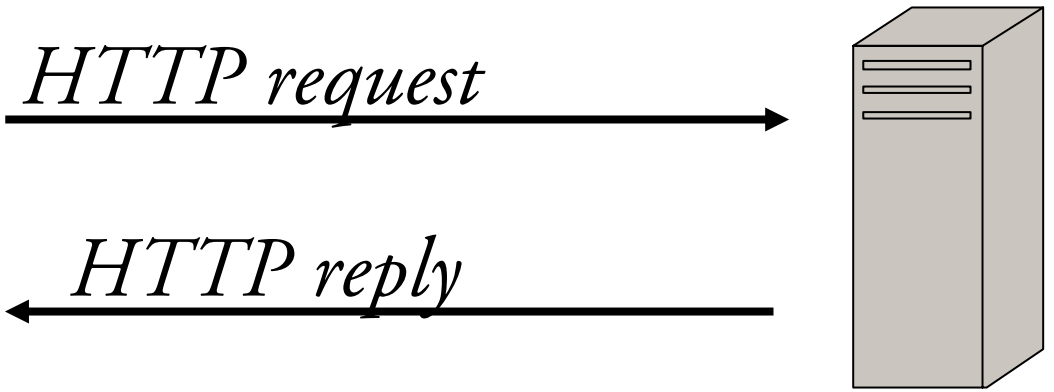
Header and trailer contains control information needed for the transmission.

Application Protocol



HTTP, an application protocol

HTTP = Hyper Text Transfer Protocol



Web server

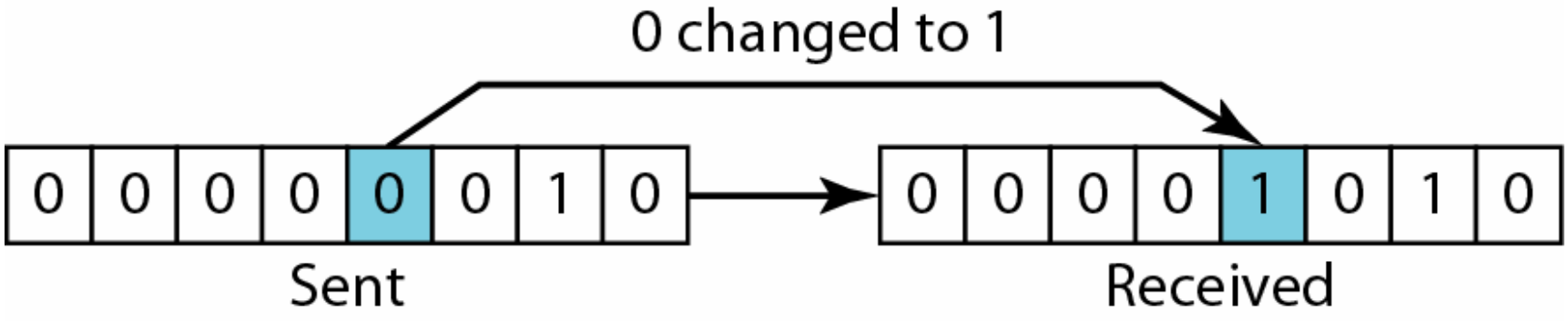
Data communication

The data packets must be received without errors.

Bit errors

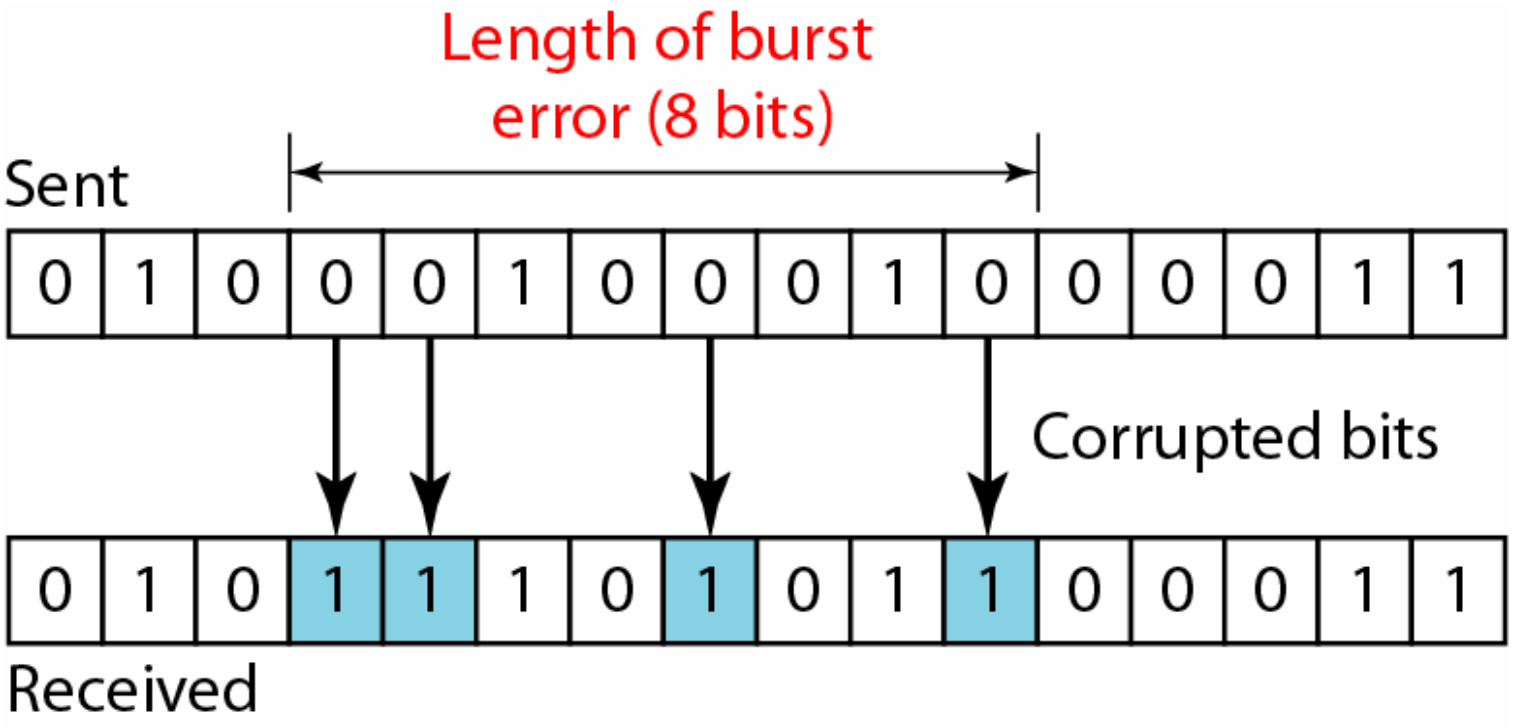
Data can be corrupted during transmission. If the receiver interprets a 1 as a 0, a bit error has occured.

Single-bit error



In a single-bit error, only 1 bit in the data unit has changed.

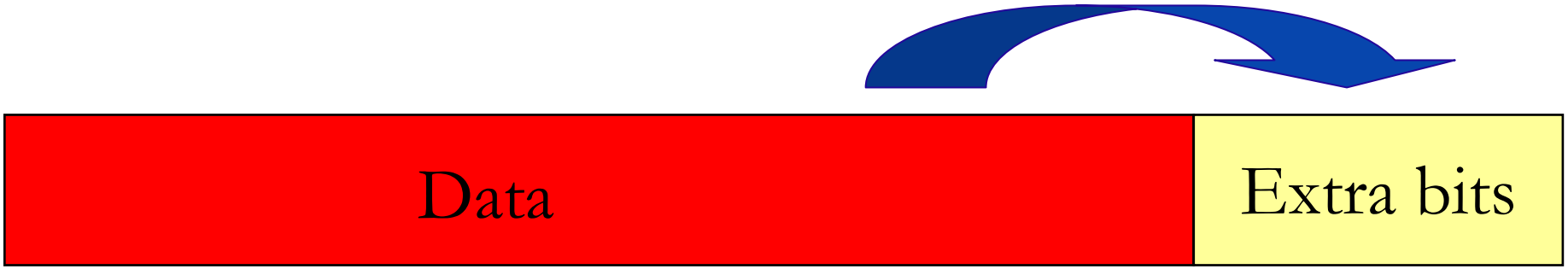
Burst errors



A burst error means that 2 or more bits in the data unit have changed.

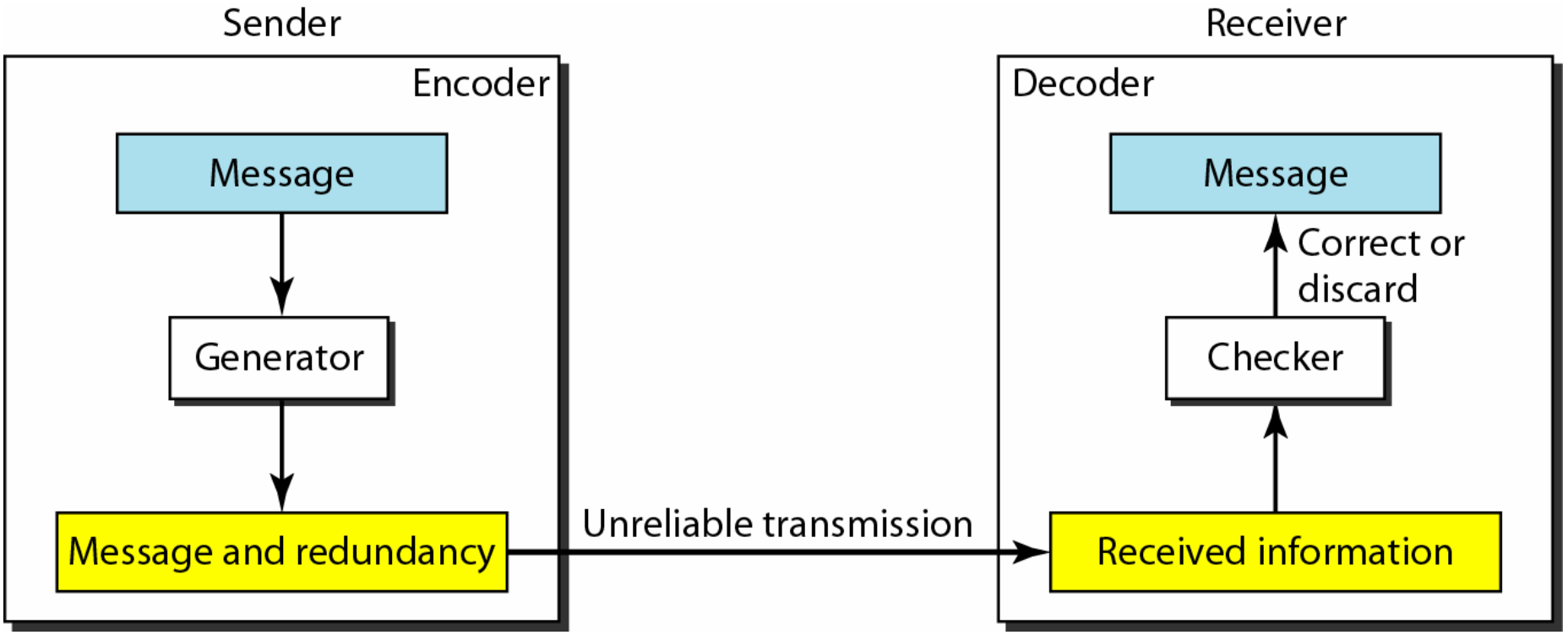
To find bit errors

To detect and/or correct errors, extra (redundant) bits is added to the data message.



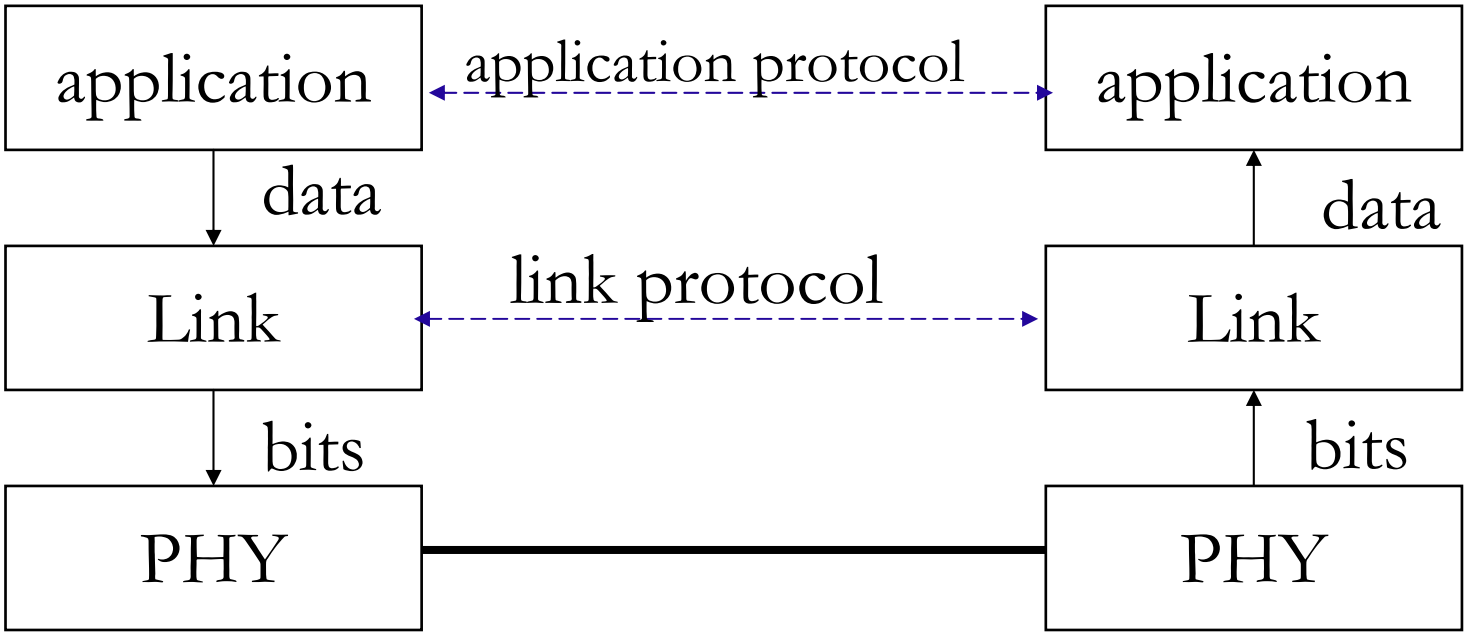
The value of the extra bits depends on the data.

Error detection process



Link layer protocol

The sender and receiver uses a link layer protocol that provides error detection and correction for data that is sent on a physical link.



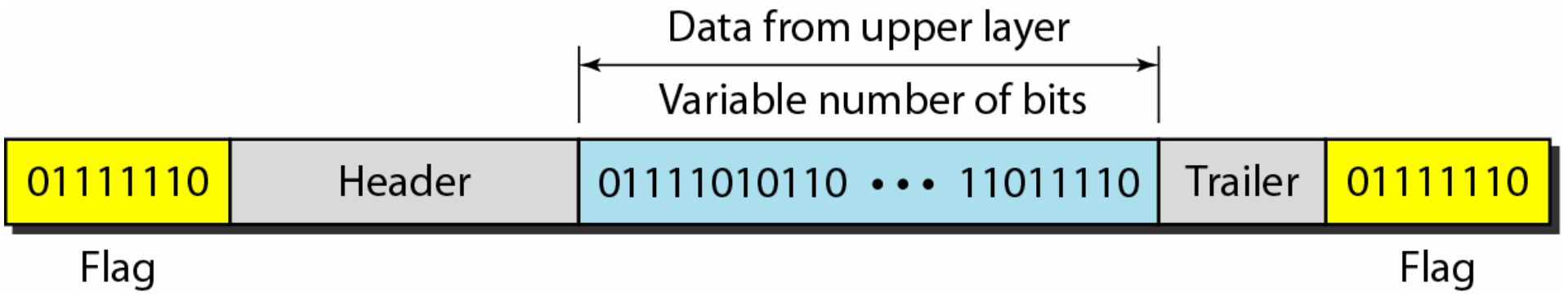
Framing

The link protocol at the sender pack data in frames so that the reciever can distinguish one frame from another.

Fixed-size frames are usually called cells. In this case, the receiver can easily separate different frames.

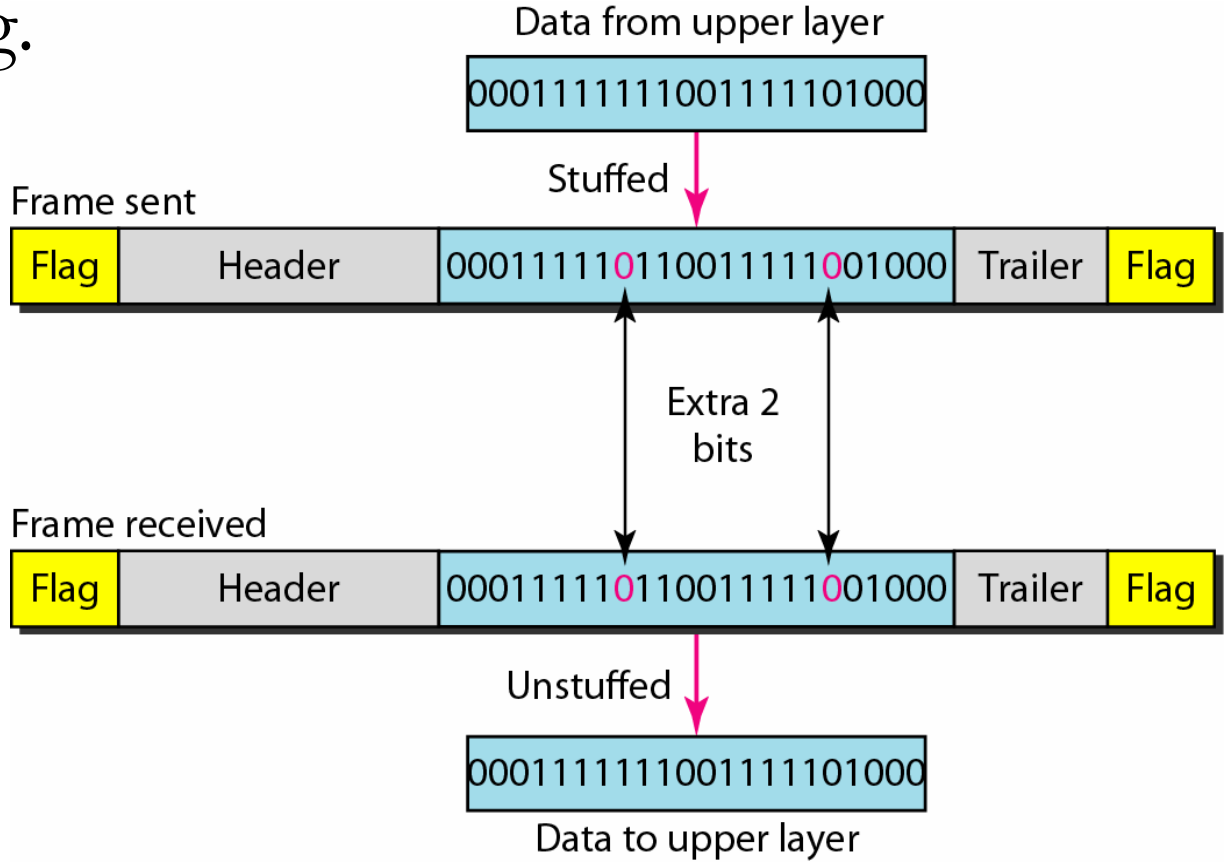
Variable-size frames need delimiters, usually called flags, so that the receiver can separate the frames.

A Variable-size frame



Bitstuffing

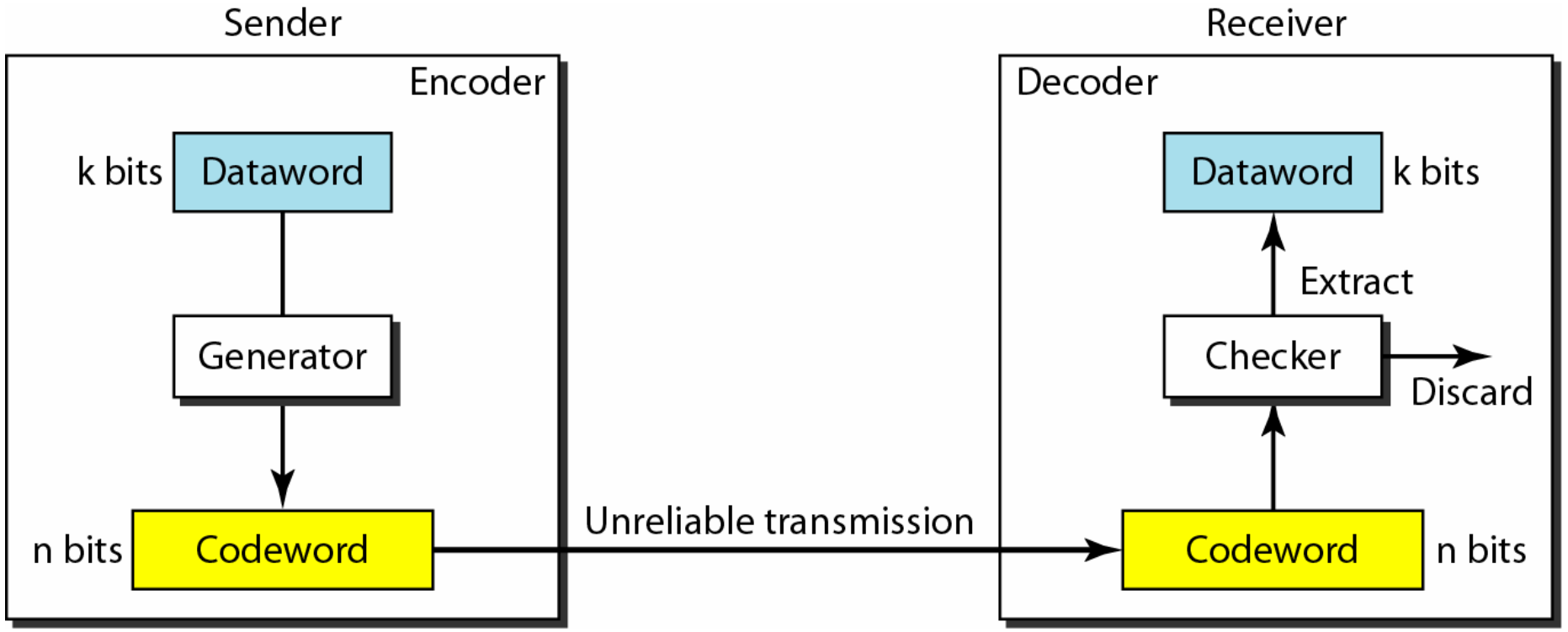
Bitstuffing is used so that the data cannot be interpreted as a flag.



Error detection with Block coding

In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length $n = k + r$. The resulting n-bit blocks are called codewords.

Block coding process



Error detection schemes

- Simple Parity-Check Code
- Cyclic Redundancy Check
- Checksum

Simple Parity-Check Code

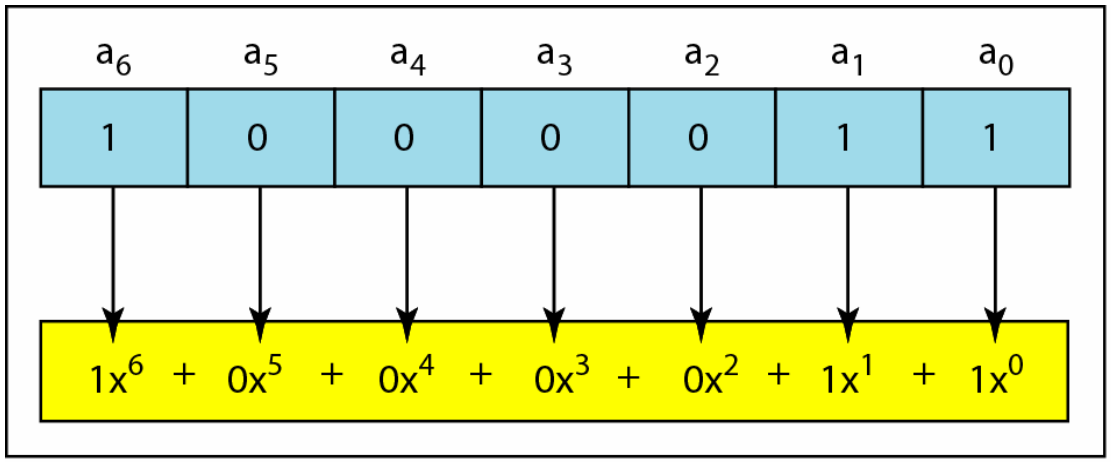
A *k*-bit dataword is changed to an *n*-bit codeword, where $n = k+1$. In (Even) Parity-Check the extra bit is selected to make the total number of 1s in the codeword even.

10011100	+	0	=	100111000

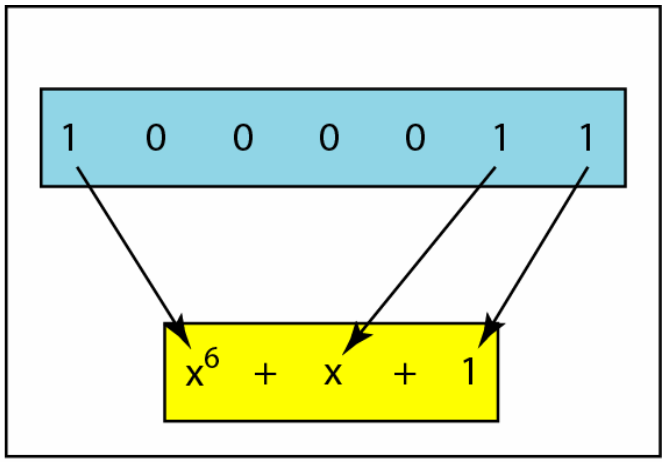
A simple parity-check code can detect an odd number of errors.

Cyclic Redundancy Check (CRC)

Let the dataword of k bits be represented by a polynomial, $d(x)$.



a. Binary pattern and polynomial



b. Short form

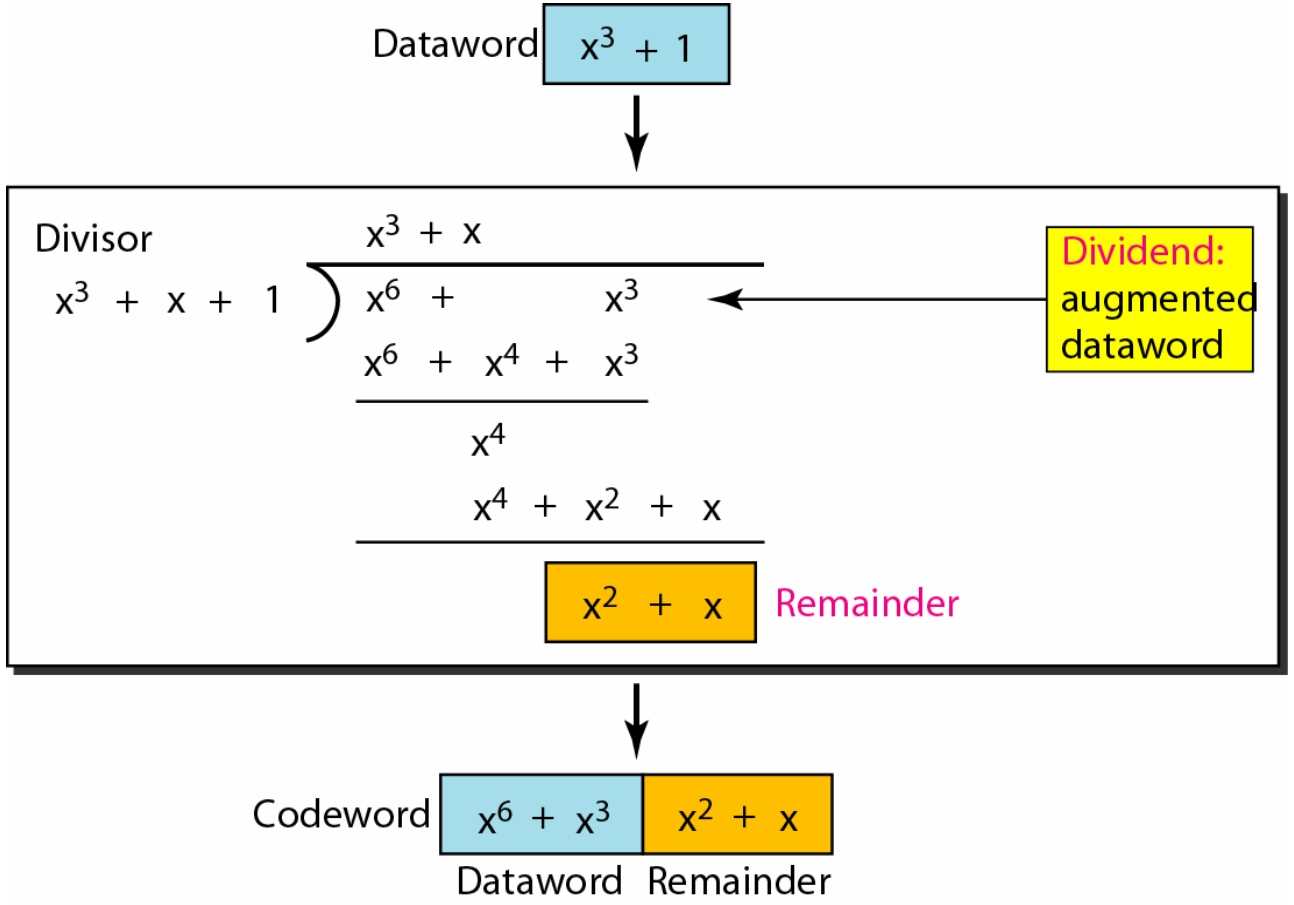
CRC (2)

- The objective is to find a codeword, $c(x)$, of n bits.
- The codeword is determined with a generator polynomial, $g(x)$, of degree n-k.
- The generator polynomial is also called the divisor.

CRC at the sender

1. Left-shift the dataword n-k bits (i.e. $x^{n-k} * d(x)$).
2. Divide $x^{n-k} * d(x)$ with $g(x)$ using binary coefficients.
3. The remainder, $r(x)$, of the division is used as redundant bits (i.e. $c(x) = x^{n-k} * d(x) + r(x)$).

CRC example


Dataword
$x^3 + 1$
Divisor
$x^3 + x + 1$
$x^3 + x$
$x^6 + x^3$
$x^6 + x^4 + x^3$
x^4
$x^4 + x^2 + x$
$x^2 + x$
Remainder
Dividend:
augmented
dataword
Codeword
$x^6 + x^3$
$x^2 + x$
Dataword
Remainder

CRC at the receiver

The received codeword is $c(x)+e(x)$ where $e(x)$ is the error that have occurred during the transmission.

The receiver calculates the following:

$$\frac{\text{Received codeword}}{g(x)} = \frac{c(x)}{g(x)} + \frac{e(x)}{g(x)}$$

If the result is 0, the dataword is assumed to be correct.

Detected errors

In a cyclic code, those $e(x)$ errors that are divisible by $g(x)$ are not caught.

If the generator has more than one term and the coefficient of x^0 is 1, all single errors can be caught.

If a generator cannot divide $x^t + 1$ (t between 0 and n – 1), then all isolated double errors can be detected.

A generator that contains a factor of $x + 1$ can detect all odd-numbered errors.

Generator polynomial

Therefore, a good polynomial generator needs to have the following characteristics:

1. It should have at least two terms.
2. The coefficient of the term x^0 should be 1.
3. It should not divide $x^t + 1$, for t between 2 and $n - 1$.
4. It should have the factor $x + 1$.

Some standard CRC polynomials

Name	*Polynomial*	*Application*
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Checksum

The checksum is used in the Internet by several protocols although not at the data link layer.

The main principle is to divide the data into segments of *n* bits. Then add the segments and use the sum as redundant bits.

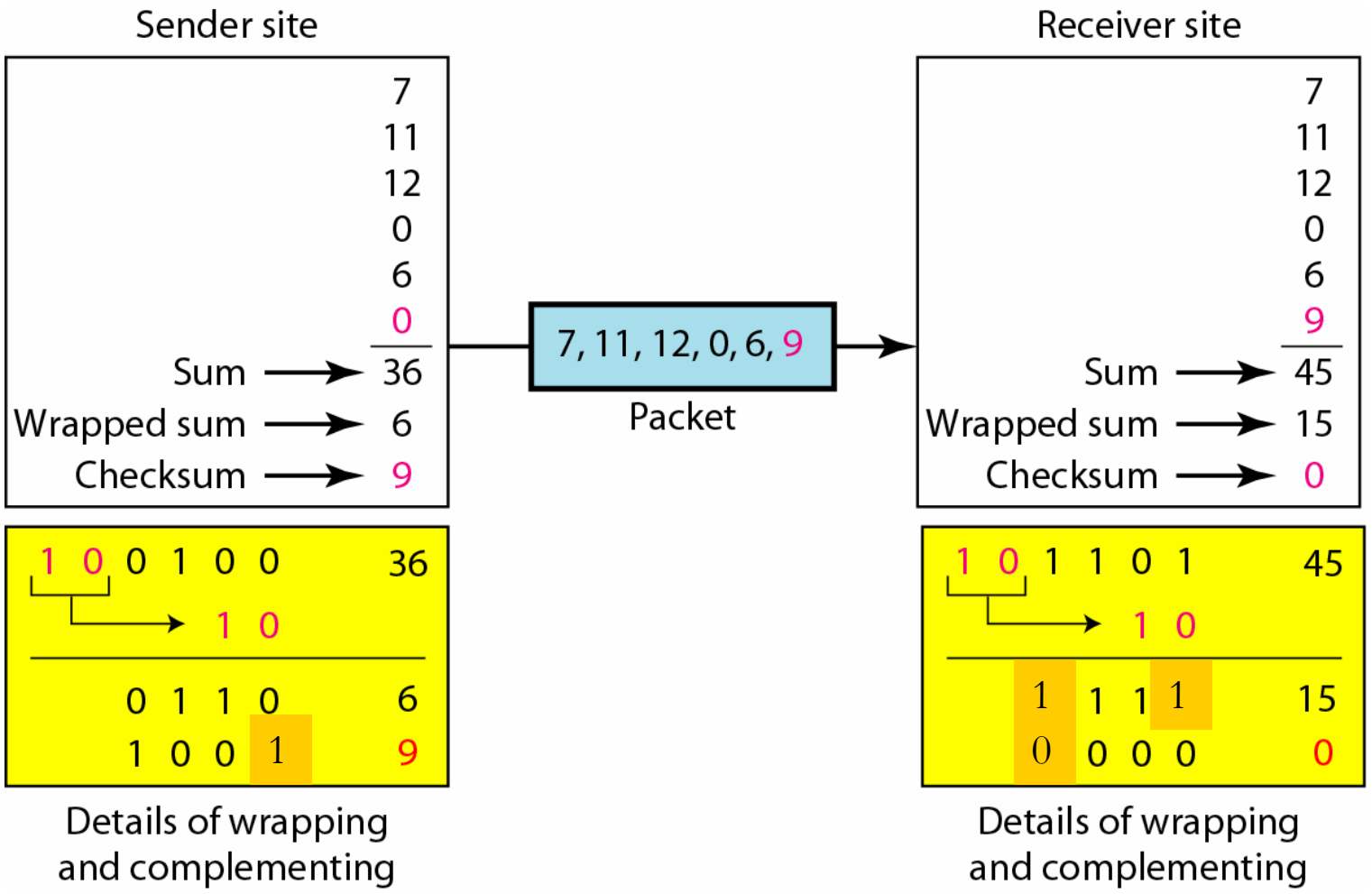
Checksum process at sender

1. The message is divided into n-bit words.
2. The value of the checksum word is set to 0.
3. All words including the checksum are added using one's complement addition.
4. The sum is complemented and becomes the checksum.
5. The checksum is sent with the data.

Checksum process at receiver

1. The message (including checksum) is divided into n-bit words.
2. All words are added using one's complement addition.
3. The sum is complemented and becomes the new checksum.
4. If the value of checksum is 0, the message is accepted; otherwise, it is rejected.

Checksum example



Error correction

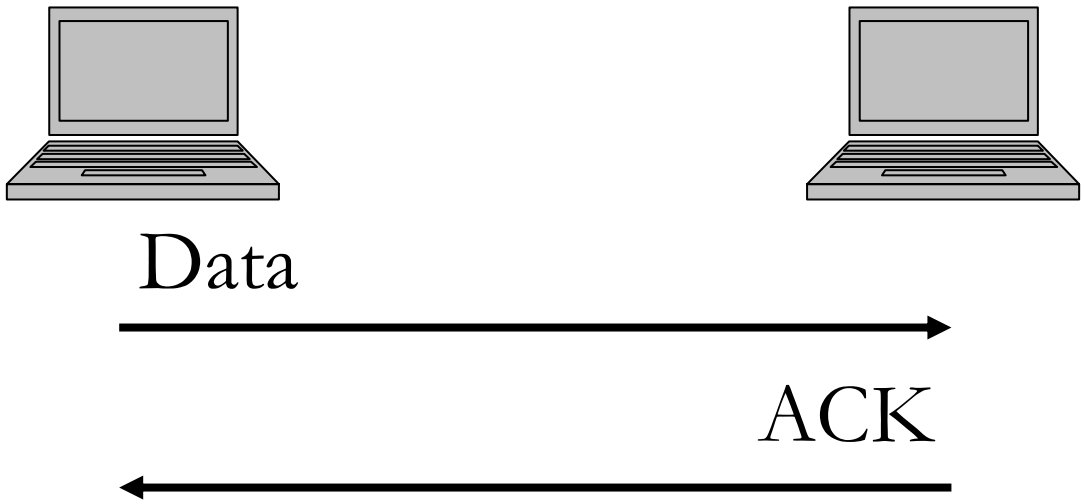
When a corrupted packet has been detected, the error must be corrected in some way.

Two basic principles:

- Forward Error Correction (FEC)
  - Extra bits are added in the data that makes it possible for the receiver to correct bit error.
- Retransmission of the data
  - The data is retransmitted until correctly received. Problem: The receiver cannot tell which data that was received corrupted.

Retransmissions

The basic principle in a retransmission scheme is that the receiver acknowledges all correctly received packets.



Automatic Repeat Request (ARQ) schemes

- Stop-and-wait ARQ
- Go-back-N ARQ
- Selective Repeat ARQ

Stop-and-wait ARQ

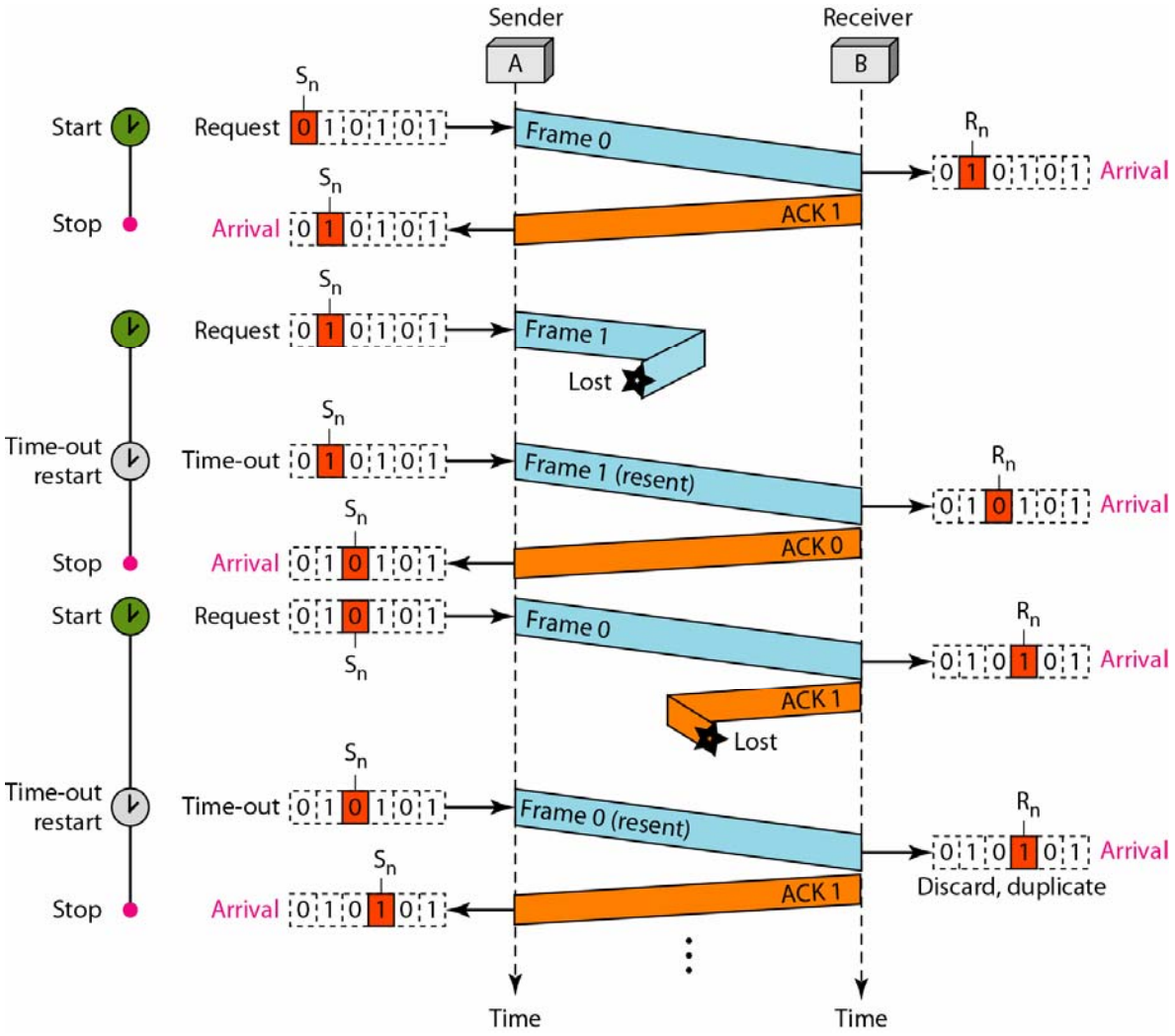
The receiver sends an ACK for every data packet that is correctly received.

The sender transmits the next packet when it has received an ACK for the previous one.

The sender uses a time-out for each packet. If the time-out expires (i.e. no ACK has arrived), the packet is retransmitted.

Packets are identified with a sequence number, alternating between 0 and 1.

Stop-and-Wait, example



Go-back-N ARQ

The sender can transmit several packets without an ACK from the receiver.

The receiver ACKs all packets that are correctly received *in the right order*.

The sender can transmit new packets when ACKs are received for previous packets.

This is called pipelining.

Sequence numbers

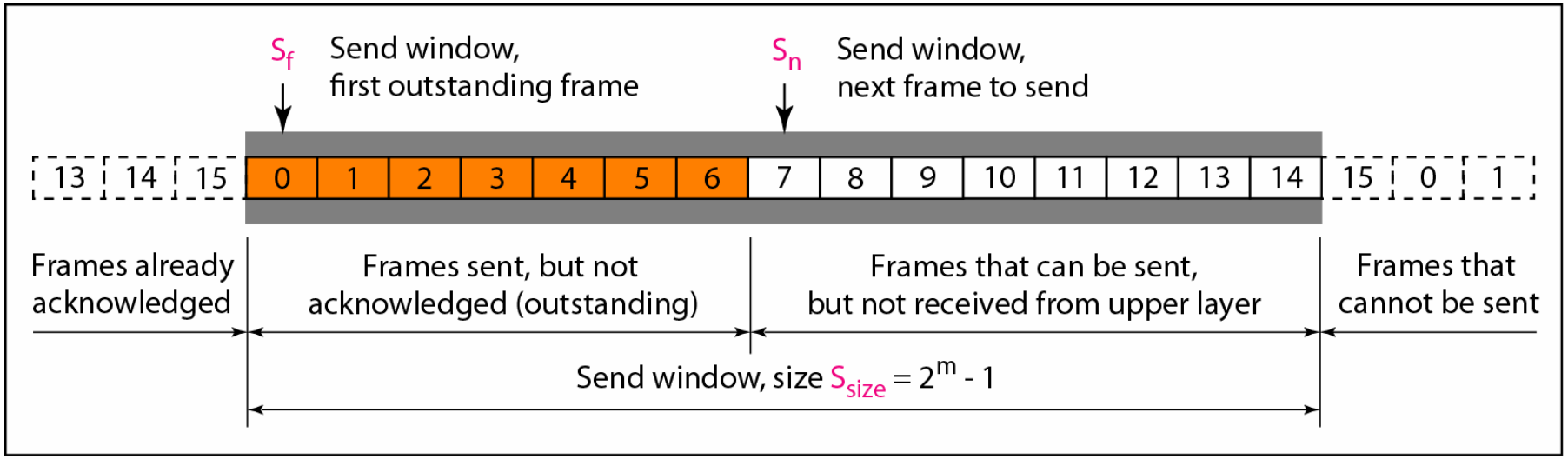
The packets are identified with a seqence number.

If the header allows m bits for the sequence number, the numbers can range from 0 to 2^m-1.

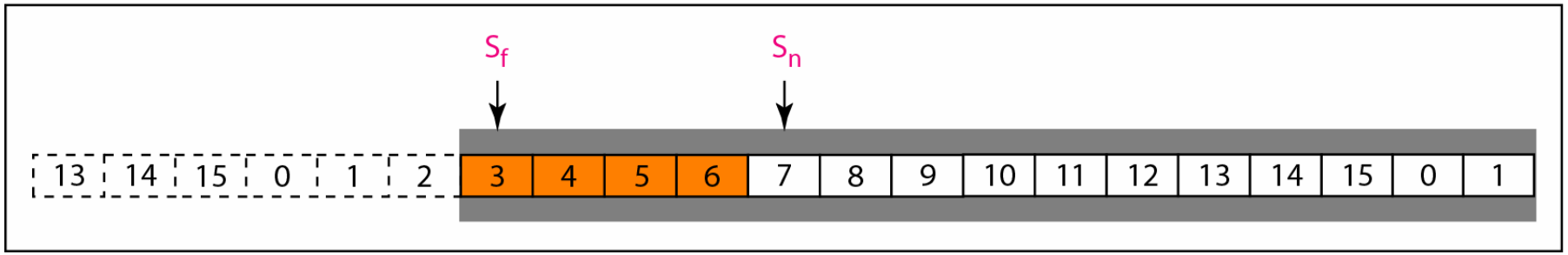
This means that the maximum number of outstanding packets (transmitted packets that haven't been ACKed) is 2^m-1.

A sliding window defines the range of sequence numbers that currently concerns the sender and receiver.

Sliding window at sender

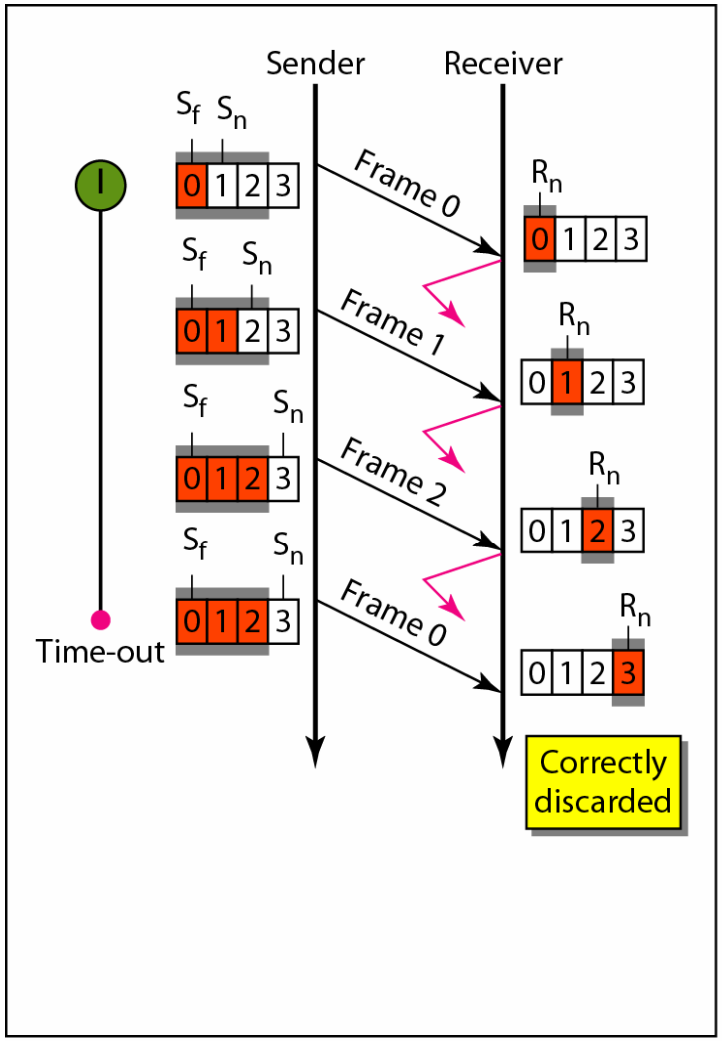


a. Send window before sliding

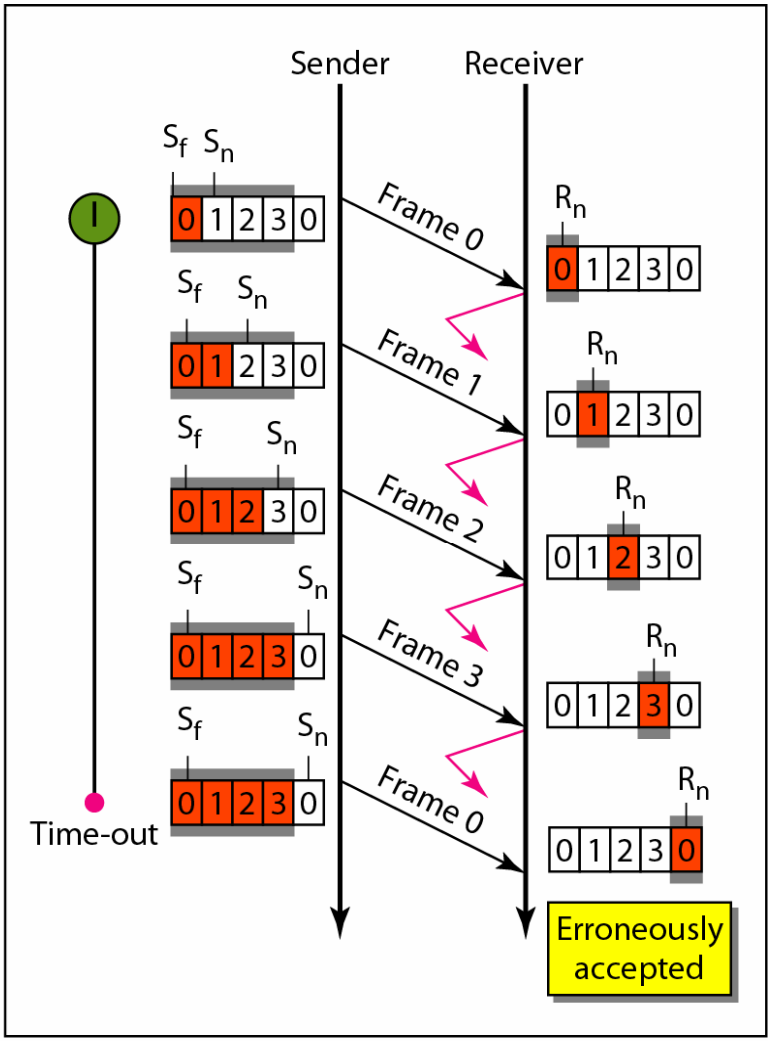


b. Send window after sliding

Importance of window size

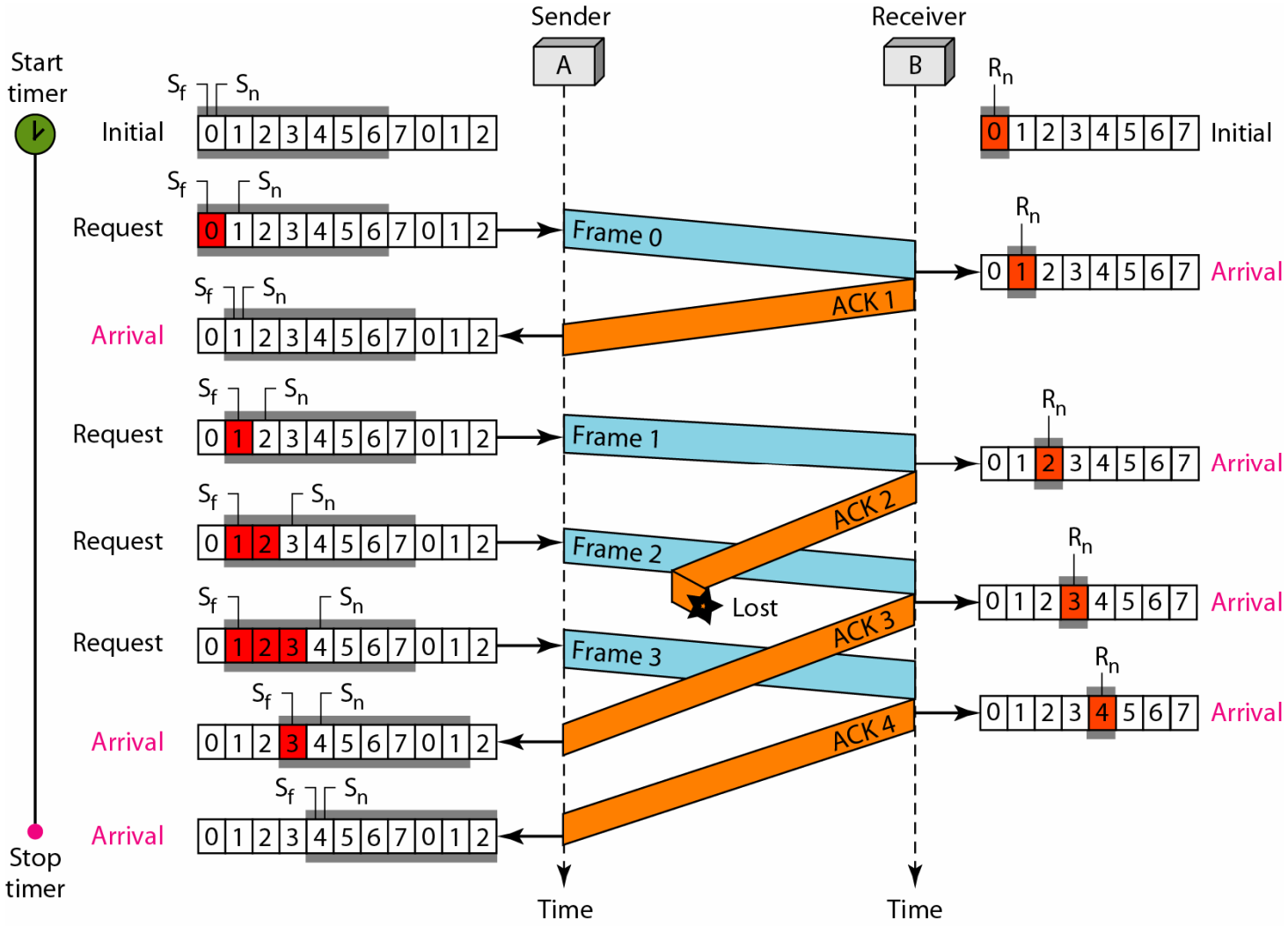


a. Window size < 2^m

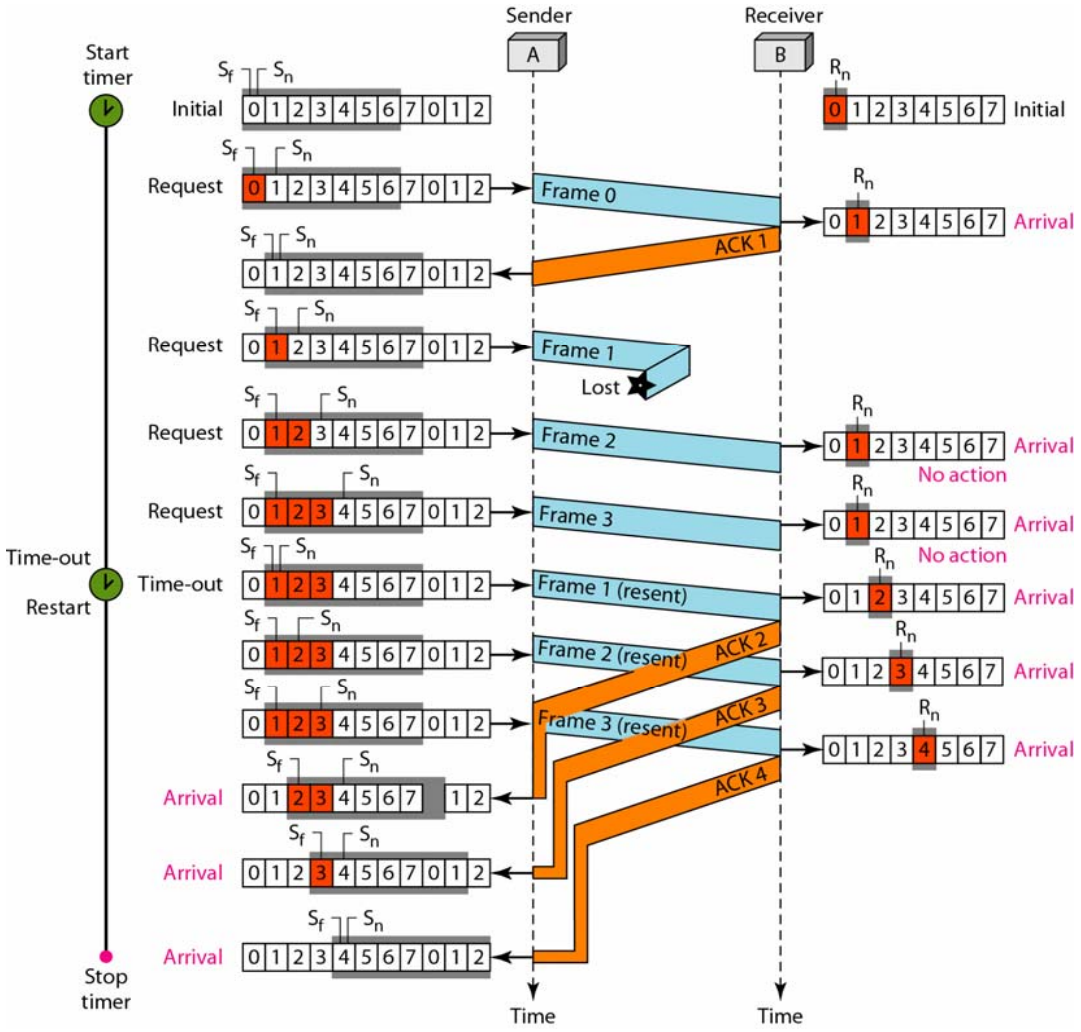


b. Window size = 2^m

Go-back-N, example (lost ACK)



Go-back-N, example (lost data)



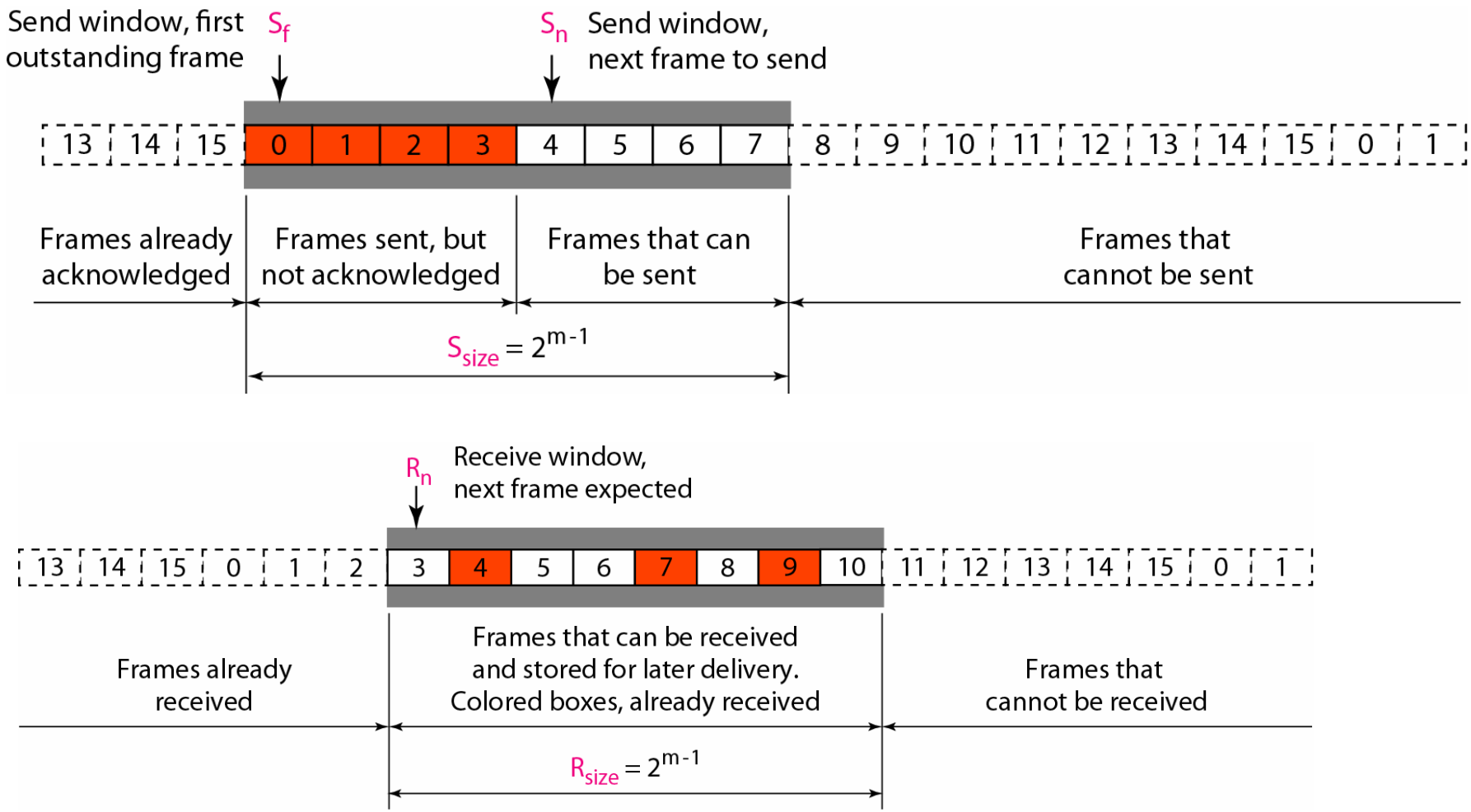
Selective repeat ARQ

Works as Go-back-N as long as there are no lost packets.

The receiver sends a negative acknowledgment (NAK) when it detects a lost packet.

Only the lost packets are retransmitted.

Send and receive windows



Selective repeat, example